

Bài 1

a)

1 (PID: 1)

1 (PID: 1)

```
|
└── WindowServer (PID: 86)
    ├── iTunes (PID: 281)
    ├── Safari (PID: 531)
    ├── Mail (PID: 726)
    └── Terminal (PID: 282)
        |
        └── login (PID: 287)
            |
            └── -bash (PID: 293)
                ├── firefox-bin (PID: 461)
                └── Aquamacs (PID: 751)b)
```

Sử dụng lệnh `ps` để kiểm tra tiến trình cha của một tiến trình thông qua PID.

Có thể dùng lệnh sau để hiển thị đầy đủ thông tin:

`ps -f -p <PID>`

Trong kết quả trả về, cột PPID chính là PID của tiến trình cha.

Nếu chỉ cần lấy riêng giá trị PPID, dùng:

`ps -o ppid= -p <PID>`

c)

Lệnh `ps tree` dùng để hiển thị cây phân cấp giữa các tiến trình.

Nếu hệ thống chưa cài đặt, có thể cài bằng lệnh:

`sudo apt install psmisc`

Để xem cây tiến trình theo một PID cụ thể và hiển thị cả PID của từng tiến trình, dùng:

pstree -s -p <PID>

Bài 2

1. Kết quả in ra màn hình: I see 17 coconuts!

Giải thích:

Khi chương trình gọi hàm `fork()`, hệ điều hành sẽ tạo thêm một tiến trình con từ tiến trình cha. Tiến trình con ban đầu nhận dữ liệu giống tiến trình cha, nhưng sau khi được tạo, mỗi tiến trình sẽ có vùng nhớ riêng biệt.

Trong đoạn chương trình, điều kiện `if(pid == 0)` tương ứng với tiến trình con. Tại đây, biến `num_coconuts` được gán lại thành 42, sau đó tiến trình con kết thúc bằng lệnh `exit(0)`. Sự thay đổi này chỉ xảy ra trong bộ nhớ của tiến trình con.

Phần `else` là đoạn mã của tiến trình cha. Tiến trình cha sử dụng `wait(NULL)` để chờ tiến trình con hoàn thành trước khi tiếp tục thực hiện.

Cuối cùng, tiến trình cha in giá trị của biến `num_coconuts` ra màn hình. Vì cha và con không dùng chung vùng nhớ cho biến cục bộ, nên việc tiến trình con đổi giá trị thành 42 không làm thay đổi giá trị trong tiến trình cha. Do đó, giá trị được in ra vẫn là 17.